3 EF Core əlaqəli məlumatı necə gətirir

Bir sifarişi bazadan aldıq. Bəs sifarişin içindəki məhsullar da gəldimi? Gəlməyibsə onları hansı sətirdə və hansı əmrlə gətirəcəyik? Bu dərs eager, explicit və lazy loading-i həmin suallar üzərində izah edir.

Eyni kafe məlumatını hər üç üsulla oxuyacağıq. Kodun yanında nəticəni və bazaya gedən SQL əmrlərinin sayını görəcəyik. Sonra çoxlu sifariş olanda yaranan N+1 problemini, lazımsız məlumatı azaltmağı və context-in ömrünü açacağıq.

Oxu ardıcıllığı belədir:

- Əvvəl cədvəl, əlaqə, entity, navigation və DbContext.
- Sonra adi sorğunun nəyi gətirdiyi və nəyi gətirmədiyi.
- Eager, explicit və lazy loading: eyni sifariş, fərqli yükləmə anı.
- Boş əlaqə ilə yüklənməmiş əlaqənin fərqi.
- N+1, projection, tracking və əvvəlki məlumatın təsiri.
- ThenInclude, bir neçə kolleksiya və split query.
- Sonda dörd fayldan ibarət tam işlək Console layihəsi.

Əsas nümunələrdə **hər təcrübə təzə DbContext ilə başlayır**. Lazy loading yalnız onun təcrübəsində aktivdir. Başqa ayarlar və əvvəlki yükləmələr nəticəni dəyişə bildiyi üçün bu şərtlər vacibdir.

SQL sayları .NET 10, EF Core 10.0.9 və real SQLite mühərriki ilə yoxlanmış nümunələrə aiddir. SQLite bu dərsdə yaddaşda işləyir, amma həqiqi SQL əmrləri icra edir. Saylar bütün layihələr üçün dəyişməz qanun kimi verilməyib.

Əvvəl bazadakı iki cədvəli görək

Cədvəl eyni quruluşlu qeydləri saxlayan bölmədir. Bir qeyd **sətir**, qeydin Id və ad kimi hissələri **sütun** adlanır.

Orders: Id	CustomerName
42	Aysel
43	Murad
44	Leyla

OrderItems: Id	OrderId	ProductName
101	42	Cay
102	42	Tort
103	43	Qehve

42 nömrəli sifarişin iki məhsulu, 43-ün bir məhsulu var. 44-ün isə məhsul sətri yoxdur. Boş sifarişi qəsdən saxlamışıq ki, yüklənmiş boş əlaqəni də yoxlaya bilək.

Əsas açar - primary key cədvəldə sətri fərqləndirir. Bu nümunədə hər iki cədvəlin öz Id sütunu var. **Xarici açar - foreign key** əlaqəli sətri göstərir: OrderItems.OrderId, Orders.Id-yə bağlanır.

Əlaqə

```
Order 42 -> OrderItem 101: Cay
         -> OrderItem 102: Tort

Order 43 -> OrderItem 103: Qehve
Order 44 -> mehsul setri yoxdur
```

Bir sifarişin çoxlu məhsul sətri ola bildiyi üçün bu, **one-to-many**, yəni birdən-çoxa əlaqədir. Məhsulun adı ayrıca katalog nümunəsi gələnədək birbaşa OrderItem-də saxlanılır.

Cədvəlin sətri proqramda entity olur

EF Core baza məlumatını C# obyektləri ilə işlətməyi asanlaşdıran kitabxanadır. **ORM - Object-Relational Mapper** obyektlərlə əlaqəli cədvəllər arasında uyğunlaşdırma edən vasitədir. EF Core belə vasitədir.

Entity EF-in baza modeli daxilində tanıdığı obyekt tipidir. Burada Order bir sifarişi, OrderItem isə sifarişin bir məhsul sətrini təmsil edir. Sinif obyektin tərifi, instance isə həmin sinifdən yaradılmış konkret nüsxədir.

C# - obyekt üzərindəki məlumatlar

```
order.Id             // Sifarisin nomresi.
order.CustomerName   // Musterinin adi.
order.Items          // Sifarisin mehsul obyektleri.

item.OrderId         // Hansı sifarişə aid olduğunu göstərir.
item.Order           // Həmin sifariş obyektinə keçid verir.
```

Property - xassə obyektin oxuya və bəzən dəyişə bildiyimiz məlumat hissəsidir. **Navigation property** əlaqəli obyektə və ya obyektlər toplusuna keçid verən xassədir.

`Order.Items` çoxlu OrderItem saxlayan **collection navigation**, yəni kolleksiya əlaqəsidir. `OrderItem.Order` tək Order-a keçən **reference navigation**, yəni tək obyekt əlaqəsidir.

Xarici açar və navigation eyni şey deyil. OrderId sadə nömrədir. Order isə əlaqəli obyektə istinaddır. **İstinad - reference** həmin obyektə çatmağa imkan verən dəyərdir.

Items üçün `new List<OrderItem>()` yazmaq boş siyahı yaradır. Bu əməliyyat özü bazaya sorğu göndərmir və məhsulları bazadan oxumur.

DbContext və provider hansı işi görür

DbContext baza sorğularını göndərən, lazım olduqda entity dəyişikliklərini izləyən EF Core obyektidir. Nümunələrdə ona db deyirik. Onun ömrü və içində əvvəl yüklənmiş məlumatlar nəticəyə təsir edə bilər.

DbSet<Order> Order entity-ləri üçün sorğu qurmağa və əlavə etmə kimi əməliyyatlara çıxış verir. `db.Orders` bazadakı bütün sifarişlərin artıq yaddaşa yüklənməsi demək deyil.

C# - sadə sorğu

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
```

`SingleAsync` dəqiq bir nəticə gözləyir. Sıfır və ya birdən çox nəticə olarsa xəta verir. `o => o.Id == 42` ifadəsi hər nəzərdən keçirilən sifariş üçün Id-nin 42 olması şərtini göstərir. == bərabərlik yoxlamasıdır.

Async - asinxron əməliyyat tamamlanmasını gözləyərkən icra axınını bloklamamağa imkan verən işləmə üsuludur. `await` həmin nəticəni gözləmək üçün yazılır. O, özü əlaqəli məlumatı yükləmək qaydası deyil.

Provider EF Core-u konkret baza sistemi ilə əlaqələndirən hissədir. SQLite, SQL Server və PostgreSQL üçün fərqli provider-lər var. **SQLite** ayrıca baza serveri tələb etmədən işləyə bilən baza mühərrikidir. Bu dərsdə asan təkrarlanan sınaq üçün seçilib.

SQLite üçün yaranan SQL-in yazılışı başqa bazada fərqlənə bilər. Əsas məntiqi öyrənir, dəqiq davranışı isə öz provider-inin SQL qeydlərində yoxlayırsan.

Sorğunu qurmaq və nəticəni almaq ayrı addımlardır

LINQ C#-da Where, Select və OrderBy kimi vasitələrlə məlumat sorğuları yazmaq imkanıdır. EF Core uyğun LINQ ifadələrini SQL-ə çevirib bazada icra edə bilir.

C# - bu hissədə hələ SQL icra olunmur

```
var query = db.Orders.Where(o => o.Id == 42);
```

`Where` nəticəyə daxil olma şərtidir. `query` hələ sorğunun təsviridir. Belə sorğu **IQueryable** müqaviləsi ilə ifadə oluna bilər: provider onu sonradan icra edə bilir.

C# - bu sətirdə sorğu icra olunur

```
var order = await query.SingleAsync();
```

Nəticəni siyahı almaq üçün `ToListAsync()` işlədilir. Sorğunun təsvirini qurub icranı nəticə tələb olunan ana saxlamaq **deferred execution - təxirə salınmış icra** adlanır.

Bu anlayış lazy loading ilə eyni deyil. Burada əsas sorğunun nə vaxt icra olunduğundan danışırıq. Lazy loading-də isə artıq alınmış entity-nin əlaqəli məlumatının sonradan xassəyə müraciətdə yüklənməsindən danışacağıq.

Materialization - obyektə çevrilmə baza nəticəsindən C# entity obyektlərinin qurulmasıdır. SQL-in nəticəsi gələndə EF uyğun Order obyektini yarada və məlumatları onun xassələrinə yerləşdirə bilər.

Əvvəlcə bu iki hadisəni ayır: **sorğunun yazılması** və **baza nəticəsinin həqiqətən alınması**.

Adi Order sorğusu Items-i avtomatik gətirmir

Lazy loading bağlıdır, context təzədir və modeldə avtomatik Include qaydası yoxdur. Belə şəraitdə yalnız sifarişi alaq:

C# - əsas nümunənin fraqmenti

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.Items.Count);
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
```

Yoxlanmış nəticə

```
0
False
SQL emrlerinin sayi: 1
```

Bazadakı sifarişin iki məhsulu var. Amma bu təcrübədə yalnız Orders sətri gətirildi. Items əvvəlcə yaratdığımız boş siyahı olaraq qaldı. **Buradakı 0 bazada məhsul olmadığını sübut etmir.**

`Count` siyahıdakı element sayıdır. `Entry(order)` EF-in həmin entity barədə məlumatına çıxış verir. `Collection(o => o.Items)` nəzarət etdiyimiz əlaqəni seçir. **IsLoaded** EF-in həmin əlaqəni yüklənmiş sayıb-saymadığını göstərir. False yanlış, True doğru məntiqi nəticəsidir.

Əvvəl başqa sorğularla məhsullar yüklənmiş olsaydı context əlaqəni qismən doldura bilərdi. Lazy loading aktiv olsaydı isə Count-a müraciət əlavə SQL işlədə bilərdi. Ona görə təcrübənin əvvəlindəki şərtləri nəzərə alırıq.

Üç loading üsulu eyni suala müxtəlif cavab verir

Loading bu dərsdə entity-nin əlaqəli məlumatının yüklənməsidir. Əsas sual budur: məhsulları nə vaxt və hansı kod addımı ilə gətirək?

Üsul	Yükləməni nə başladır	Sifariş ekranından nümunə
Eager	Əsas sorğuda Include ilə əvvəlcədən tələb edilir	Detallar ekranında məhsulların lazım olacağı məlumdur.
Explicit	Sonradan ayrıca Load əmri verilir	İstifadəçi detalları açanda əlaqə açıq əmrlə alınır.
Lazy	Yüklənməmiş əlaqənin xassəsinə müraciət edilir	Kod Items-i oxuyanda əlaqə avtomatik alınır.

Eager “əvvəlcədən”, explicit “açıq şəkildə”, lazy isə “ehtiyac anına saxlanmış” mənasında düşünülə bilər.

Eyni əlaqə, fərqli başlanma anı

```
Eager:    Order + Items iste -> sorqunu icra et.

Explicit: Order al -> sonra Items ucun Load emri ver.

Lazy:     Order al -> Items-e toxun -> gizli yukleme islesin.
```

Eager loading-in tərifi “həmişə bir SQL” deyil. Split query seçiləndə əlaqəni əvvəlcədən tələb edən sorğu bir neçə SQL əmrinə bölünə bilər. Əsas fərq əlaqənin hansı mərhələdə tələb edilməsidir.

İndi hər üsulu eyni 42 nömrəli sifarişdə yoxlayaq.

Eager loading: Items-i əsas nəticə ilə birlikdə tələb et

C# - eager loading

```
var order = await db.Orders
    .Include(o => o.Items)
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.Items.Count);
```

`Include(o => o.Items)` əsas sifariş nəticəsi hazırlanarkən Items əlaqəsinin də doldurulmasını istəyir. Burada entity siniflərini dəyişmədik; sorğuya əlaqə tələbini əlavə etdik.

Bu təcrübənin nəticəsi

```
Sifaris: 42
Mehsullar: Cay, Tort
Items.Count: 2
IsLoaded: True
SQL: 1
```

Sifariş alındıqdan sonra Items-i gəzmək və Count-u oxumaq əlavə SQL göndərmir. Bu nümunədə tək kolleksiya bir SQL əmri ilə gətirilir.

SQL baza ilə işləmək üçün sorğu dilidir. Əmr sayı bu dərsdə hazırlıq və ilkin məlumat əlavə etmə bitəndən sonra ölçülür. Sayğaca yalnız EF-in icra olunmuş baza əmri qeydləri daxil edilir.

Təzə context istifadə etdiyimiz üçün Items-in əvvəlki sorğudan hazır qalması nəticəyə təsir etmir. İstifadəçi həmişə sifariş və məhsullarını birlikdə görəcəksə, bu, oxunması aydın seçimdir.

Eager SQL-də cədvəlləri necə birləşdirə bilər

JOIN uyğunluq şərtinə görə müxtəlif cədvəllərin sətirlərini bir nəticədə birləşdirən SQL əməliyyatıdır. Eager nümunəsinin məntiqi aşağıdakı sadələşdirilmiş SQL ilə görünür:

SQL - izah üçün sadələşdirilmiş görünüş

```
SELECT o.Id, o.CustomerName, i.Id, i.ProductName
FROM Orders AS o
LEFT JOIN OrderItems AS i ON o.Id = i.OrderId
WHERE o.Id = 42;
```

`AS` cədvələ qısa ad verir. `ON` əlaqələndirmə şərtidir. `LEFT JOIN` soldakı sifarişi uyğun məhsul sətri olmasa da saxlaya bilir. Tam EF SQL-də əlavə sütunlar, sıralama və SingleAsync üçün məhdudlaşdırma da görünə bilər.

Order Id	CustomerName	Item Id	ProductName
42	Aysel	101	Cay
42	Aysel	102	Tort

SQL nəticəsində sifariş məlumatı iki sətirdə təkrarlanır. EF bunu bir Order və onun iki OrderItem obyekti kimi qura bilir. **Bir SQL nəticəsində çox sətir olması çox SQL əmri getməsi demək deyil.**

44 nömrəli sifarişin məhsulu yoxdur. Eager loading bu sifarişi yenə gətirə və Items-i yüklənmiş boş kolleksiya kimi hazırlaya bilər.

Tam işlək proqramın sonunda SQL qeydləri də ekrana yazılır. Orada sadələşdirilmiş nümunəni deyil, provider-in həqiqətən icra etdiyi SQL-i görə bilərsən.

Explicit loading: əlaqəni ayrıca əmrlə yüklə

Lazy loading bağlıdır. Əvvəl sifarişi, sonra açıq əmrlə məhsullarını alırıq:

C# - explicit loading

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();

Console.WriteLine(order.Items.Count);
```

`LoadAsync` seçilmiş əlaqəni asinxron yükləyir. Yükləmənin başladığı sətir kodda açıq görünür.

Bu təcrübənin addımları

```
Order alinir.          SQL cemi: 1
LoadAsync isleyir.     SQL cemi: 2
Items.Count oxunur.    SQL cemi: 2, say: 2
```

Əsas sorğu Orders-dan 42-ni alır. İkinci sorğu OrderItems-dan OrderId-si 42 olan sətirləri gətirir. Cavab yenə Cay və Tort-dur, amma bu dəfə iki ayrı əmrlə alındı.

İstifadəçi detalları açmasa və Load əmri verilməsə, ikinci sorğu getməz. Üstünlük həmin qərarı açıq idarə etməkdir. Xərc əlavə baza müraciətidir.

Bu təcrübədə əlaqə tam yükləndikdən sonra eyni Collection üçün təkrar LoadAsync əlavə SQL göndərmir; IsLoaded artıq True olur. Filtrlənmiş və qismən əlaqələrdə bu işarənin mənasını ayrıca açacağıq.

Collection çoxluğu Reference isə tək əlaqəni seçir

Items çoxlu element saxladığı üçün Collection istifadə etdik. İndi 101 nömrəli məhsul sətrindən onun sifarişinə keçək. Burada Order tək əlaqədir:

C# - reference explicit loading

```
var item = await db.OrderItems
    .SingleAsync(i => i.Id == 101);

await db.Entry(item)
    .Reference(i => i.Order)
    .LoadAsync();

Console.WriteLine(item.Order.Id);
```

Nəticə

```
42
SQL emrlerinin sayi: 2
```

Birinci sorğu OrderItem 101-i gətirir. Onun OrderId dəyəri 42-dir. İkinci sorğu əlaqəli Order 42-ni gətirir və item.Order keçidini doldurur.

Əlaqə	Seçən əməliyyat
Order.Items: çoxlu məhsul sətri	Collection
OrderItem.Order: tək sifariş	Reference

Reference burada EF-in tək əlaqə ilə işləmək üçün verdiyi əməliyyatın adıdır. Obyektə istinad anlayışı ilə əlaqəlidir, amma ayrıca new əmri deyil.

Explicit loading üçün əlaqəni idarə edən DbContext hələ istifadə edilə bilən vəziyyətdə olmalıdır. Bağlanmış context ilə yeni baza əmri göndərə bilmərik.

Lazy loading üçün əvvəlcə mexanizmi aktivləşdiririk

Adi entity-də Items xassəsinin olması lazy loading-i öz-özünə aktiv etmir. Bu dərsdə **proxy** üsulunu istifadə edirik. Proxy xassəyə müraciəti tutub əlavə davranış yerinə yetirən vasitəçi obyekt kimi işləyir.

C# - context ayarlarının qurulduğu yerdə

```
var builder = new DbContextOptionsBuilder<CafeContext>()
    .UseSqlite(connection)
    .UseLazyLoadingProxies();
```

`UseSqlite` baza provider-ini seçir. `UseLazyLoadingProxies` lazy proxy mexanizmini açır. Bunun üçün Microsoft.EntityFrameworkCore.Proxies paketi lazımdır. Paket layihəyə əlavə olunan hazır kitabxanadır. Tam layihənin hazırlanması sonda verilir.

C# - Order daxilində əlaqə xassəsi

```
public virtual ICollection<OrderItem> Items { get; set; }
    = new List<OrderItem>();
```

virtual xassənin davranışının törəmə sinifdə əvəz edilə bilməsini bildirir. Törəmə sinif başqa sinfin quruluşunu genişləndirən sinifdir. Proxy bunu istifadə etdiyindən entity sinfindən törəmək mümkün olmalıdır; sealed ilə bağlanmış sinif uyğun deyil.

ICollection<OrderItem> OrderItem obyektləri toplusunu ifadə edən müqavilədir. `List<OrderItem>` həmin toplunun konkret reallaşdırmasıdır. `get` oxuma, `set` dəyər yazma imkanını göstərir.

ILazyLoader ilə başqa üsul da mövcuddur: yükləmə köməkçisi entity-yə verilir və xassə onu çağırır. Onun tələbləri proxy ilə eyni deyil. Bu dərsin işlək nümunəsi bütövlükdə proxy üsulundan istifadə edir.

Lazy loading-də adi Items oxunuşu SQL işlədə bilər

Proxy mexanizmi aktivdir, context təzədir. Əsas sorğuda Include yazmırıq:

C# - lazy loading

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.Items.Count);
Console.WriteLine(order.Items.Count);
```

Bu təcrübənin addımları

```
Order alinir.             SQL cemi: 1
Items ilk defe oxunur.    SQL cemi: 2, say: 2
Items tekrar oxunur.      SQL cemi: 2, say: 2
```

İlk `order.Items` müraciətində proxy əlaqənin yüklənmədiyini görür və məhsulları bazadan alır. Count sonra həmin kolleksiyanın sayını oxuyur. Yüklənməmiş bütöv kolleksiyaya müraciət etdiyimiz üçün burada yalnız say deyil, məhsul obyektləri də gətirilir.

İkinci müraciətdə əlaqə artıq yüklənib və ayrıca SQL lazım olmur. Items-ə heç müraciət etməsəydik, bu təcrübədə ikinci sorğu getməzdi.

Explicit variantda əlavə SQL-i LoadAsync sətri başladırdı. Lazy variantda həmin iş xassə oxunuşunun arxasında baş verir. Gözə sadə görünən kodun baza ilə işləməsi əsas fərqdir.

Debug zamanı obyektin xassələrini açmaq da getter-ə müraciət edib lazy yükləməyə səbəb ola bilər. Getter xassə oxunanda işləyən hissədir. Müşahidə edərkən SQL qeydlərinə də baxmaq ona görə faydalıdır.

Eyni məlumat üçün üç üsulun nəticəsini müqayisə et

Bu cədvəl bir Order 42, bir Items kolleksiyası, təzə context və göstərilən ayarlar üçündür. Hər üsulda sonda iki məhsul alırıq.

Mərhələ	Eager	Explicit	Lazy
Əsas Order sorğusundan sonra SQL sayı	1	1	1
Həmin anda Items yüklənmiş sayılır	Bəli	Xeyr	Xeyr
Əlavə yükləməni başlayan	Include artıq tələb edib	LoadAsync çağırışı	İlk Items oxunuşu
Items hazır olduqda ümumi SQL sayı	1	2	2
Nəticədə məhsul sayı	2	2	2

Explicit və lazy-də son SQL sayı eyni ola bilər. Bu onların eyni üsul olması demək deyil: yükləmə qərarının hansı sətirdə və hansı mexanizmlə verilməsi fərqlənir.

Eager nümunəsində bir SQL olması həmin sadə sorğunun nəticəsidir. Eager ilə AsSplitQuery istifadə etsək, əvvəlcədən tələb olunan əlaqə bir neçə SQL ilə gətirilə bilər.

Bir üsulun iki SQL, digərinin bir SQL işlətməsi də sürətin dəqiq nisbətini vermir. Məlumat həcmi, təkrar sütunlar, əlaqə gecikməsi və bazanın sorğunu necə icra etməsi də təsir edir.

Sualı belə ver: bu ekran hansı məlumatı nə vaxt istəyir və onu gətirmək hansı ölçülən xərci yaradır?

Boş kolleksiya ilə yüklənməmiş kolleksiya fərqlidir

44 nömrəli sifarişin bazada məhsulu yoxdur. Lazy loading bağlı olan təzə context-də əvvəl yalnız sifarişi alırıq:

C# - boş əlaqə təcrübəsi

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 44);
var relation = db.Entry(order).Collection(o => o.Items);

Console.WriteLine(order.Items.Count);
Console.WriteLine(relation.IsLoaded);

await relation.LoadAsync();

Console.WriteLine(order.Items.Count);
Console.WriteLine(relation.IsLoaded);
```

Nəticə

```
0
False
0
True
```

Əvvəlki 0 sadəcə yaddaşdakı siyahının boşluğudur. Sonrakı 0 isə bu tam yükləmə nümunəsində bazaya baxıldıqdan sonra da əlaqəli sətir tapılmadığını göstərir. Ümumi SQL sayı 2-dir.

IsLoaded EF-in yükləmə işarəsidir. Sonra göstərəcəyimiz filtered Include-də əlaqə qismən tələb olunsa da True ola bilər. Ona görə True dəyərini hər şəraitdə “bazadakı bütün mümkün sətirlər buradadır” kimi oxuma.

N+1 problemini üç sifarişlə sayaq

Lazy loading aktivdir. Əvvəl bütün sifarişləri, sonra hərəsinin məhsullarını oxuyuruq:

C# - lazy loading ilə dövr

```
var orders = await db.Orders
    .OrderBy(o => o.Id)
    .ToListAsync();

foreach (var order in orders)
{
    Console.WriteLine(order.Items.Count);
}
```

`OrderBy` nəticəni göstərilən dəyərə görə sıralayır. `foreach` hər element üçün daxildəki kodu işləyir. Burada Items-in ilk oxunuşu hər sifarişdə ayrıca lazy sorğu başladır.

Addım	Yeni SQL	Ümumi SQL
Üç sifarişi al	1	1
42-nin Items əlaqəsini aç	1	2
43-ün Items əlaqəsini aç	1	3
44-ün boş Items əlaqəsini yoxla	1	4

Ekrana 2, 1, 0 yazılır. Sifariş sayı 3 olsa da SQL sayı 4-dür: `1 + N`. Buna **N+1 problemi** deyilir. N bu təcrübədə ayrıca əlaqəsi yüklənən sifarişlərin sayıdır.

Boş əlaqə üçün də bazaya baxmaq lazım oldu. Eyni model 100 sifarişdə təkrarlansa 101 SQL yarana bilər. Əlavə hər müraciət baza ilə gediş-gəliş xərci daşıyır. Əvvəlki yükləmələr və başqa ayarlar dəqiq sayı dəyişə bilər.

Explicit dövrü də N+1 yarada bilər

Əlavə əmrin kodda açıq görünməsi onun sayını azaltmır. Lazy loading bağlı olsa da aşağıdakı üsul üç sifariş üçün dörd SQL göndərir:

C# - explicit loading ilə dövr

```
var orders = await db.Orders.OrderBy(o => o.Id).ToListAsync();
foreach (var order in orders)
{
    await db.Entry(order).Collection(o => o.Items).LoadAsync();
}
```

Bütün sifarişlərin məhsulları lazım olduğunu əvvəlcədən biliriksə, əlaqəni ümumi sorğuda tələb edə bilərik:

C# - həmin ekran üçün eager alternativ

```
var orders = await db.Orders
    .Include(o => o.Items)
    .AsSingleQuery()
    .OrderBy(o => o.Id)
    .ToListAsync();
```

`AsSingleQuery` bu sorğunun əlaqələrini bir SQL əmri ilə gətirməyi açıq seçir. Bu kiçik nümunədə say 1-dir. Gətirilən Order-ların məhsul sayları yenə 2, 1, 0-dır.

N+1 yalnız lazy loading-in adı deyil. Əsas məsələ hər əsas entity üçün ayrıca baza müraciəti təkrarlanmasıdır. Həll də ekranın real ehtiyacını müəyyən edib əlaqəni əvvəlcədən və ya toplu gətirməkdir.

Bir SQL-də həddindən çox təkrar məlumat yaranarsa split query seçimi də nəzərdən keçirilə bilər. Onu iki kolleksiyanın nümunəsində ayrıca ölçəcəyik.

Yalnız say lazımdırsa bütün Items obyektlərini gətirmə

Siyahı ekranında yalnız sifariş nömrəsi və məhsul sayı lazımdır. Nəticənin formasını birbaşa Select ilə qura bilərik. Buna **projection - lazım olan məlumatın seçilməsi** deyilir.

C# - bütün sifarişlər üçün say nəticəsi

```
var rows = await db.Orders
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.Id,
        ItemCount = o.Items.Count
    })
    .ToListAsync();
```

`new { ... }` adı ayrıca elan edilməmiş sadə nəticə tipi qurur. Bu ifadədə Count SQL-də hesablanır; bütün OrderItem obyektlərini əvvəl C# yaddaşına gətirmək lazım olmur. Bu nümunədə **bir SQL** və 42 üçün 2, 43 üçün 1, 44 üçün 0 sayı alınır. Include tələb olunmur.

Artıq bir Order alınıbsa, ayrıca onun əlaqəsinin sayını da bazadan soruşmaq olar:

C# - əlaqə sorğusunda say

```
var count = await db.Entry(order)
    .Collection(o => o.Items)
    .Query()
    .CountAsync();
```

`Query` həmin əlaqə üçün ayrıca sorğu təsviri verir. `CountAsync` bazada sayı hesablayır. Əvvəlki Order sorğusu ilə birlikdə burada 2 SQL var, amma Items kolleksiyası yüklənmir və IsLoaded False qalır.

Bu ilə lazy `order.Items.Count` fərqlidir: lazy nümunədə Items getter-i əvvəl bütün kolleksiyanı yükləyirdi. Eyni Count sözü harada işlədiyinə görə fərqli iş yarada bilər.

Loading və tracking ayrı qərarlardır

Loading əlaqəli məlumatı nə vaxt gətirəcəyimizi müəyyən edir. **Tracking** isə gətirilən entity-lərin dəyişikliklərini DbContext-in izləməsidir. Dəyişikliklər SaveChanges zamanı bazaya yazıla bilər.

C# - eager, amma əsas dəyişiklik izləyicisinə əlavə etmir

```
var order = await db.Orders
    .AsNoTracking()
    .Include(o => o.Items)
    .SingleAsync(o => o.Id == 42);
```

Burada Items yenə yüklənir və sayı 2 olur. Amma bu nəticələr context-in əsas dəyişiklik izləyicisinə əlavə edilmir.

C# - həmin no-tracking nəticəsində

```
order.CustomerName = "Yeni ad";
await db.SaveChangesAsync();
```

Obyektin dəyərini dəyişib sadəcə SaveChanges çağırmaq bu halda dəyişikliyi avtomatik bazaya yazmır. Nümunədə yazılan entity sayı 0 olur. Eyni əməliyyat adi tracking sorğusunun nəticəsində edildikdə isə dəyişiklik saxlanılır.

AsNoTracking lazy loading-i söndürmək düyməsi deyil. Bu dərsdə yoxlanmış EF Core 10.0.9 proxy nümunəsində no-tracking nəticəsində də context açıq ikən Items oxunuşu lazy SQL işlədə bilir. Ona görə gizli yükləməni dayandırmaq üçün təkcə tracking seçiminə güvənmə.

Əvvəlki entity surətlərini izləmək əlavə iş yarada bilər, amma no-tracking üçün sabit faizlə sürət vədi yoxdur. Nəticənin həcmi, təkrar obyektlər və istifadə məqsədi nəzərə alınır.

Əvvəlki sorğular navigation-u qismən doldura bilər

Lazy loading bağlıdır. Order-u alıb sonra onun məhsullarından yalnız birini ayrıca soruşaq:

C# - eyni tracking context daxilində

```
var order = await db.Orders.SingleAsync(o => o.Id == 42);
await db.OrderItems.Where(i => i.Id == 101).ToListAsync();

Console.WriteLine(order.Items.Count);
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
```

Nəticə

```
1
False
SQL: 2
```

EF OrderItem.OrderId ilə Order.Id-nin uyğun olduğunu görüb obyektlər arasındakı əlaqəni qurur. Buna **navigation fixup - əlaqəli istinadların uyğunlaşdırılması** deyilir.

Items-də bir obyekt var, amma bazada ikinci məhsul da mövcuddur. **Kolleksiyada məlumat olması onun tam yüklənməsi demək deyil.** Bu təcrübədə IsLoaded False qalır.

Sonra həmin Collection üçün LoadAsync çağırsaq, qalan əlaqəli məlumat da alınır. Items.Count 2, SQL sayı 3 olur.

Identity resolution eyni açarlı entity üçün eyni obyekt nüsxəsinin istifadə edilməsidir. Adi tracking bunu təmin edə bilər. Fixup isə uyğun obyektlər arasında navigation-ları bağlayır. Bu iki işi yeni SQL göndərilməsi ilə qarışdırma: fixup artıq gətirilmiş obyektləri əlaqələndirə bilir.

Filtered Include qismən əlaqəni yüklənmiş saya bilər

Filter nəticəni şərtlə məhdudlaşdırmaqdır. Bu nümunədə 42-nin yalnız 101 nömrəli məhsul sətrini istəyirik:

C# - təzə tracking context ilə

```
var order = await db.Orders
    .Include(o => o.Items.Where(i => i.Id == 101))
    .SingleAsync(o => o.Id == 42);
```

Items.Count 1 olur. SQL sayı 1-dir. Amma EF bu əlaqəni artıq yüklənmiş sayır: IsLoaded True olur. Adi təkrar LoadAsync çağırışı bu təcrübədə qalan məhsulu özü gətirmir.

Eyni context-də əvvəldən hər iki məhsulu oxusaydıq, fixup səbəbindən filtrlənmiş Include-dən sonra Items-də iki obyekt görünə bilərdi:

C# - başqa, əvvəldən məlumatlı context ssenarisi

```
await db.OrderItems
    .Where(i => i.OrderId == 42)
    .ToListAsync();

var order = await db.Orders
    .Include(o => o.Items.Where(i => i.Id == 101))
    .SingleAsync(o => o.Id == 42);
```

Bu ssenaridə Items.Count 2 alınır. Filtr SQL-də yerinə yetirilir, amma context-in əvvəl izlədiyi uyğun obyektlər də navigation-a birləşdirilə bilir.

Filtrlənmiş nəticəni dəqiq, ayrıca oxuma görünüşü kimi istəyirsənsə təzə context, uyğun no-tracking sorğusu və ya projection düşün. **IsLoaded True dəyərini həmişə “bazadakı bütün sətirlər gətirilib” kimi şərh etmə.**

Lazy loading context-in ömründən asılıdır

DI dərsində scope-un iş sərhədi olduğunu gördük. Adi ASP.NET Core HTTP tətbiqində AddDbContext ilə qeyd edilən DbContext standart olaraq scoped olur. HTTP scope-unu EF deyil, ASP.NET Core açır.

Context bağlandıqdan sonra hələ yüklənməmiş əlaqəni lazy üsulla açmağa çalışmaq problemdir:

C# - context artıq bağlanır

```
Order order;
await using (var db = new CafeContext(lazyOptions))
{
    order = await db.Orders.SingleAsync(o => o.Id == 42);
}

var count = order.Items.Count;
```

`lazyOptions` proxy aktiv olan context ayarlarını bildirir. Bu yoxlanmış nümunədə son sətir bağlanmış context-də lazy yükləmə cəhdi barədə xəta verir. `using` blokdan çıxışda resursu təmizləyir; `await using` asinxron təmizlənməni gözləyə bilir.

Əlaqə əvvəlcədən yüklənibsə, sadə obyekt məlumatını oxumaq ayrıca baza çağırışına ehtiyac yaratmaya bilər. Məsələ yeni yükləmə üçün işlək context lazım olmasıdır.

Entity-ni birbaşa şəbəkə cavabına çevirmək də xassələrin oxunmasına səbəb olur. **Serialization** obyektin JSON kimi ötürülən formata çevrilməsidir. Lazy əlaqələr bu anda gözlənilməz SQL işlədə bilər. İki tərəfli Order və Items keçidləri dövr də yarada bilər.

DTO - Data Transfer Object cavabda ötürüləcək məlumat üçün ayrıca sadə tipdir. Projection ilə yalnız lazım olan dəyərləri seçib cavab formasını açıq qurmaq bu davranışı idarə etməyi asanlaşdırır.

Async əsas sorğunu gözləməkdir lazy getter ayrıca işdir

C# - proxy aktiv olanda

```
var order = await db.Orders.SingleAsync(o => o.Id == 42);
var count = order.Items.Count;
```

Birinci sətirdəki await yalnız əsas sorğunu gözləyir. İkinci sətirdə yüklənməmiş Items-ə müraciət proxy lazy loading-ni başladanda ayrıca sinxron baza çağırışı işləyə bilər. **Sinxron** çağıran icra axınının iş tamamlanana qədər gözləməsi deməkdir.

Əlavə əlaqənin yüklənməsini açıq asinxron gözləmək istəyiriksə explicit LoadAsync və ya əvvəlcədən Include ilə uyğun asinxron əsas sorğu istifadə edə bilərik.

C# - əlavə yükləmə açıq görünür

```
await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
```

Eyni DbContext-də eyni anda iki baza əməliyyatını başlatmaq uyğun deyil. Məsələn, həmin context-də iki ayrıca sorğunu Task.WhenAll ilə paralel işlətmək doğru seçim olmur. Task əməliyyatın tamamlanmasını təmsil edən obyektdir; WhenAll bir neçə belə tamamlanmanı gözləməyə imkan verir.

Əvvəl bir əməliyyatı await et, sonra növbətisinə keç. Həqiqətən ayrıca paralel işlər lazımdırsa onların context və resurs sərhədləri də ayrıca qurulmalıdır.

Bu dərsin SQL sayğacı da ardıcıl təcrübələr üçündür. Yanaşı işləyən sorğuların qeydlərini eyni siyahıda qarışdırıb bir sorğunun xərci kimi təqdim etmirik.

ThenInclude əlaqənin növbəti pilləsini seçir

İndi modeli genişləndirək: OrderItem-in ayrıca məhsul katalogundakı Product-a keçidi olsun. Bu əlavə hissələr əsas iki-cədvəlli layihəyə dəyişiklikdir; onların test məlumatını sonrakı səhifədə verəcəyik.

C# - yeni Product.cs

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
}
```

C# - OrderItem sinfinə əlavə et

```
public int? ProductId { get; set; }
public virtual Product? Product { get; set; }
```

`int` tam ədəd tipidir. `int?` dəyərin boş ola bilməsini, Product? isə əlaqəli obyektin olmaya bilməsini göstərir. CafeContext-ə `public DbSet<Product> Products => Set<Product>();` əlavə edilir.

C# - iki pilləli əlaqənin yüklənməsi

```
var order = await db.Orders
    .Include(o => o.Items)
    .ThenInclude(i => i.Product)
    .SingleAsync(o => o.Id == 42);
```

Yol belədir: **Order -> Items -> Product**. Include birinci əlaqəni, ThenInclude həmin əlaqənin ardını seçir. Bu genişləndirilmiş model və göstərilən məlumatla məhsul adları Cay və Tort, SQL sayı 1 alınır.

ThenInclude və ikinci Include eyni yol demək deyil. İkinci Include yenidən əsas Order-dan başqa bir əlaqə seçə bilər. Növbəti nümunə bunu göstərəcək.

İkinci Include ayrıca kolleksiyanı seçir

Sifarişin məhsulları ilə yanaşı ayrıca ödəniş qeydləri də olsun. Bunun üçün Payment entity-si və Order-da ikinci kolleksiya əlavə edirik:

C# - yeni Payment.cs

```
public class Payment
{
    public int Id { get; set; }
    public int OrderId { get; set; }
    public int Amount { get; set; }
    public virtual Order Order { get; set; } = null!;
}
```

C# - Order sinfinə əlavə et

```
public virtual ICollection<Payment> Payments { get; set; }
    = new List<Payment>();
```

CafeContext-ə `public DbSet<Payment> Payments => Set<Payment>();` əlavə edilir. Amount sadə nümunədə ödəniş məbləğini bildirir.

C# - əsas sifarişdən iki ayrı əlaqə

```
var order = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSingleQuery()
    .SingleAsync(o => o.Id == 42);
```

Burada yollar **Order -> Items** və **Order -> Payments**-dir. Bunlar eyni səviyyədə iki kolleksiyadır. Product-a gedən əvvəlki nümunədə isə yol Items-in içindən davam edirdi.

Növbəti məlumat əlavə etmə addımı ilə Order 42-də iki məhsul və üç ödəniş olacaq. Hər iki kolleksiyanı bir SQL-də birləşdirməyin nəticəsini sonra sayacağıq.

Genişləndirilmiş modelin test məlumatını əlavə edək

Product, Payment və əvvəlki iki səhifədəki xassələr əlavə olunandan sonra bu SeedExtras.cs faylını yaz. `SeedData.CreateAsync(options)` çağırışından sonra `await SeedExtras.AddAsync(options);`, ondan sonra isə `sql.Clear();` işlət.

C# - SeedExtras.cs

```
using Microsoft.EntityFrameworkCore;
public static class SeedExtras
{
    public static async Task AddAsync(DbContextOptions<CafeContext> options)
    {
        await using var db = new CafeContext(options);
        db.Products.AddRange(
            new Product { Id = 1, Name = "Cay" },
            new Product { Id = 2, Name = "Tort" });
        db.Payments.AddRange(
            new Payment { Id = 1, OrderId = 42, Amount = 5 },
            new Payment { Id = 2, OrderId = 42, Amount = 10 },
            new Payment { Id = 3, OrderId = 42, Amount = 15 });
        await db.SaveChangesAsync();

        var tea = await db.OrderItems.SingleAsync(i => i.Id == 101);
        var cake = await db.OrderItems.SingleAsync(i => i.Id == 102);
        tea.ProductId = 1;
        cake.ProductId = 2;
        await db.SaveChangesAsync();
    }
}
```

Bu əlavə yalnız genişləndirilmiş model üçündür. Əsas üç loading üsulunu göstərən layihə onu tələb etmir. Nümunənin yaddaş bazası hər yeni işə salınmada təzə qurulur. Mövcud daimi bazanın strukturunu dəyişmək isə ayrıca model və migration idarəsi tələb edir. Migration baza strukturuna dəyişiklikləri tətbiq etmək üçün addımlar toplusudur.

İki kolleksiya SQL nəticəsində 2 x 3 sətir yarada bilər

Order 42-də iki məhsul və üç ödəniş var. Eyni SQL nəticəsində hər məhsul hər ödənişlə qoşulduqda altı birləşmə yaranır:

Məhsul	Ödəniş Id
Cay	1
Cay	2
Cay	3
Tort	1
Tort	2
Tort	3

Bu, **cartesian explosion - birləşmiş nəticənin vurularaq böyüməsi** problemidir. İki eyni səviyyəli kolleksiya üçün nəticə 2 + 3 deyil, bu nümunədə 2 x 3 olur.

EF entity açarlarını nəzərə alıb nəticədə yenə iki OrderItem və üç Payment obyekti qura bilir. Buna baxmayaraq SQL nəticəsində təkrarlanan sətirlər və sütunlar daşınıb.

Yoxlanmış AsSingleQuery nümunəsində bir EF SQL əmri, altı uyğun JOIN sətri, iki məhsul və üç ödəniş alınır. **C# obyektlərinin sayı ilə SQL-in sətir sayını ayır.**

Eyni problem hər ThenInclude zəncirində avtomatik yaranmır. Əlaqələrin eyni səviyyədə ayrı kolleksiyalar olması mühümdür. Bir məhsulun öz alt əlaqəsinə keçmək başqa quruluşdur.

AsSplitQuery eager sorğunu bir neçə SQL-ə bölür

C# - genişləndirilmiş modeldə

```
var order = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery()
    .SingleAsync(o => o.Id == 42);
```

Bu nümunədə üç SQL işləyir: əsas sifariş, Items və Payments. İki kolleksiyanın sətirləri eyni nəticədə vurulub altıya çevrilmir. Items.Count yenə 2, Payments.Count yenə 3 olur.

Yoxlanmış sorğu	SQL sayı
Bir kolleksiya Include + AsSingleQuery	1
Bir kolleksiya Include + AsSplitQuery	2
İki eyni səviyyəli kolleksiya + AsSingleQuery	1
İki eyni səviyyəli kolleksiya + AsSplitQuery	3

Split query dördüncü loading növü deyil. Əlaqəni əvvəlcədən tələb etdiyimiz eager sorğunun SQL əmrlərinə bölünmə seçimidir.

Qazanc təkrarlanan böyük nəticənin azalması ola bilər. Xərc əlavə baza gediş-gəlişi və bəzi nəticələrin aralıq yaddaşda saxlanmasıdır. Bir neçə SQL arasında başqa əməliyyat məlumatı dəyişə bilər. Eyni anın məlumatı tələb olunursa uyğun transaction və oxuma təcridi düşünülür.

Transaction bir qrup baza işinin razılaşdırılmış sərhədidir; təcrid isə başqa əməliyyatların dəyişikliklərinin necə görünəcəyini müəyyən edir. Split query təkbaşına belə sabit oxuma zəmanəti vermir. Dəqiq seçim real məlumat həcmi və ölçmə ilə edilir.

Səhifələmə ilə əlaqə filtrini ayır

Pagination - səhifələmə böyük nəticəni hissələrlə almaqdır. Əvvəl sifarişləri seçib sıralayır, sonra həmin hissənin əlaqələrini gətiririk:

C# - ilk iki sifariş və onların məhsulları

```
var orders = await db.Orders
    .OrderBy(o => o.Id)
    .Take(2)
    .Include(o => o.Items)
    .AsSplitQuery()
    .ToListAsync();
```

`Take(2)` əsas nəticədən iki sifariş götürür. Bu təcrübədə 42 və 43, onların məhsulları və iki SQL əmri alınır.

C# - bu isə başqa məhdudlaşdırmadır

```
var orders = await db.Orders
    .Include(o => o.Items.Where(i => i.Id == 101))
    .ToListAsync();
```

İkinci nümunə əsas sifarişlərin sayını iki etmir; hər sifariş üçün gətirilən Items əlaqəsini filtrləyir. Əsas sifarişləri azaltmaq istəyirsənsə Where və Take əsas sorğuya tətbiq edilməlidir.

Səhifələmədə sabit və tam fərqləndirən sıralama seç. Burada əsas açar Id istifadə etdik. Təkcə tarix kimi təkrarlanan dəyərə görə sıralama müxtəlif səhifələrin sərhədini qeyri-müəyyən edə bilər.

EF versiyası və provider bəzi SQL formalarını fərqli yaradır. Xüsusilə split və səhifələmə birlikdə işlənəndə nəticəni öz layihənin versiyası ilə yoxlamaq lazımdır.

Hansı ekran üçün hansı başlanğıc daha aydındır

Ehtiyac	Düşünüləcək yanaşma
Sifariş detalları həmişə məhsullarla göstərilir	Lazım olan əlaqələri Include ilə əvvəlcədən istə.
Məhsullar yalnız bəzi hallarda açılır	Explicit yükləmə qərarını ehtiyac yaranan yerdə ver.
Xassə oxunuşunun baza işi yaratması qəbul edilir	Lazy ayarlarını, context ömrünü və SQL sayını nəzarətdə saxla.
Yalnız nömrə, ad və say lazımdır	Projection ilə cavab məlumatını birbaşa seç.
Bir neçə böyük kolleksiya nəticəni vuraraq böyüdür	Single və split variantlarını həqiqi məlumatda ölç.

Seçim “ən qısa kod hansıdır?” sualına görə edilmir. Hansı məlumatın lazım olduğu, nə vaxt tələb edildiyi, neçə gediş-gəliş və nə qədər məlumat yarandığı nəzərə alınır.

Əsas nümunələrdə avtomatik əlaqə yükləmə qaydası qurmadıq. EF modelində **AutoInclude** əlaqənin sorğularda avtomatik daxil edilməsini istəyə bilər. Bu da sorğunu oxuyarkən görünməyən əlavə yükləmə səbəblərindən biridir.

İstehsal tətbiqində SQL sayını məhdud testdə ölçüb sonra real nəticə həcmi, cavab müddəti və baza yükü ilə birlikdə qiymətləndir. Bir SQL olması təkbaşına ucuz olması demək deyil.

Ən çox qarışdırılan məqamlar

“Items boşdur, deməli bazada məhsul yoxdur.” Əlaqə yüklənməmiş ola bilər. Lazy aktivdirsə oxunuş özü yükləməni başlada bilər. IsLoaded və SQL qeydləri ilə hansı mərhələdə olduğuna bax.

“Eager həmişə bir SQL-dir.” Əlaqəni əvvəlcədən istəmək loading qərarıdır. AsSplitQuery onun bir neçə SQL ilə gətirilməsini seçə bilər.

“Explicit yazmışamsa N+1 olmaz.” Dövrdə hər entity üçün ayrıca Load çağırırsansa N+1 yenə yarana bilər.

“AsNoTracking yazdım, əlaqə artıq yüklənməz.” Tracking və loading ayrı seçimlərdir. Yoxlanmış proxy nümunəsində no-tracking entity-də də lazy yükləmə işləyir.

“Items-də bir obyekt var, əlaqə tamdır.” Fixup və filtrlənmiş nəticə yalnız bir hissəni gətirmiş ola bilər. IsLoaded işarəsini də seçilmiş sorğu ilə birlikdə şərh et.

“Əvvəlki context-də nəticə belə idi, yenisində niyə fərqlidir?” Köhnə context-in izlədiyi obyektlər əlaqələri doldurmuş ola bilər. Müqayisə üçün təzə context və aydın ayarlar istifadə et.

“Single ilə First eyni yoxlamadır.” SingleAsync dəqiq bir nəticə gözləyir. FirstAsync isə ilk nəticəni götürür və birdən çox uyğun nəticəni təkbaşına xəta saymır. Hər ikisi nəticə olmadıqda xəta verə bilər; OrDefault variantları boş nəticəyə imkan verir.

“Lazy-ni await etmişdim.” Əsas sorğunun await olunması sonradan xassəyə müraciətdə yaranan işi avtomatik asinxron etmir.

Tam işlək layihəni hazırlayaq

Bu hissədən sonrakı dörd C# faylı əsas üç loading üsulunu eyni məlumat üzərində işlədən tam Console layihəsidir. Əlavə Product və Payment hissələrini qoşmadan başla. Git-də hazır nüsxə **EF_Core_Loading_Demo** qovluğundadır.

Özün yaratmaq üçün .NET 10 SDK ilə terminalda yaz:

Terminal əmrləri

```
dotnet new console -n EfLoadingDemo -f net10.0
cd EfLoadingDemo
```

SDK layihəni qurmaq və işlətmək üçün alətlər toplusudur. Yaranan EfLoadingDemo.csproj faylını aşağıdakı ilə əvəz et. csproj layihənin hədəfini və istifadə etdiyi paketləri bildirən fayldır.

XML - EfLoadingDemo.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.9" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Proxies" Version="10.0.9" />
    <PackageReference Include="SQLitePCLRaw.bundle_e_sqlite3" Version="3.0.5" />
  </ItemGroup>
</Project>
```

Paket versiyaları nümunənin təkrar yoxlana bilməsi üçün sabitlənib. Sqlite provider-i bazanı, Proxies lazy mexanizmini, SQLitePCLRaw paketi isə nümunədə istifadə edilən yerli SQLite hissəsini təmin edir. XML-də PackageReference kitabxana asılılığını göstərir.

Models.cs entity və əlaqə təriflərini saxlayır

Layihəyə Models.cs əlavə et. Bu fayldakı iki sinif əvvəl göstərdiyimiz iki cədvəli və onların əlaqəsini ifadə edir.

C# - Models.cs, tam fayl

```
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public virtual ICollection<OrderItem> Items { get; set; }
        = new List<OrderItem>();
}

public class OrderItem
{
    public int Id { get; set; }
    public int OrderId { get; set; }
    public string ProductName { get; set; } = "";
    public virtual Order Order { get; set; } = null!;
}
```

Id və OrderId adlarına əsasən EF bu sadə əsas və xarici açar əlaqəsini razılaşdırılmış ad qaydaları ilə müəyyən edir. Bu nümunədə ayrıca model qurma əmri tələb olunmur.

`CustomerName = ""` ilkin olaraq boş mətn verir. `new List<OrderItem>()` boş kolleksiya qurur. `null!` isə başlanğıcdakı boş istinad üçün kompilyator xəbərdarlığını susdurur; Order obyekti yaratmır. Naviqasiyanın doldurulması əlaqənin yüklənməsindən və fixup-dan asılıdır.

Virtual navigation-lar proxy sınağı üçün hazırdır. Onların virtual olması təkbaşına lazy loading açmır; həmin mexanizm ayrıca options-da seçilir.

CafeContext.cs bazaya çıxış nöqtələrini saxlayır

Layihəyə CafeContext.cs əlavə et:

C# - CafeContext.cs, tam fayl

```
using Microsoft.EntityFrameworkCore;

public class CafeContext : DbContext
{
    public CafeContext(DbContextOptions<CafeContext> options)
        : base(options)
    {
    }

    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();
}
```

`: DbContext` CafeContext-in EF-in əsas context sinfindən törədiyini göstərir. Konstruktor xaricdən ayarlar dəsti alır. `base(options)` onları baza sinfinin konstruktoruna verir.

DbContextOptions<CafeContext> bu context üçün provider və digər davranış ayarlarını daşıyır. Beləliklə, entity təriflərinə baxmadan konfiqurasiyanı ayrıca seçə bilirik.

`Orders => Set<Order>()` Order entity-ləri ilə işləyən DbSet-i qaytaran qısa xassə yazılışıdır. Buradakı `=>` nəticəni qaytaran ifadəni göstərir. Eyni məntiq OrderItems üçün işləyir.

Bu xassələrin olması bütün baza sətirlərini əvvəlcədən yükləmir. Hələ sorğu qurulmalı və icra edilməlidir. Context-ə çıxış ilə məlumatın yaddaşda hazır olmasını ayrı düşün.

SeedData.cs hər sınağın başlanğıc məlumatını yaradır

Seed data test və ya ilkin qurulum üçün əlavə edilən başlanğıc məlumatıdır. Bu fayl üç sifarişi və üç məhsul sətrini yaradır:

C# - SeedData.cs, tam fayl

```
using Microsoft.EntityFrameworkCore;

public static class SeedData
{
    public static async Task CreateAsync(
        DbContextOptions<CafeContext> options)
    {
        await using var db = new CafeContext(options);
        await db.Database.EnsureCreatedAsync();

        db.Orders.AddRange(
            new Order { Id = 42, CustomerName = "Aysel" },
            new Order { Id = 43, CustomerName = "Murad" },
            new Order { Id = 44, CustomerName = "Leyla" });

        db.OrderItems.AddRange(
            new OrderItem
            { Id = 101, OrderId = 42, ProductName = "Cay" },
            new OrderItem
            { Id = 102, OrderId = 42, ProductName = "Tort" },
            new OrderItem
            { Id = 103, OrderId = 43, ProductName = "Qehve" });

        await db.SaveChangesAsync();
    }
}
```

`static` bu köməkçi əməliyyatın ayrıca SeedData obyekti yaratmadan sinif adı ilə çağırılmasına imkan verir. AddRange bir neçə entity-ni əlavə etmək üçün qeyd edir. SaveChangesAsync həmin dəyişiklikləri bazaya yazır. EnsureCreatedAsync təzə nümunə bazasının strukturunu yaradır.

Bu proqram hər yeni işə salınmada yaddaş bazasını təzədən açır. Ona görə başlanğıc məlumatı əvvəlki işə salınmadan qalmır. Bu hazırlıq mərhələsinin SQL-ləri əsas loading sayına daxil edilmir.

Program.cs üç variantdan birini seçib ölçür

`args` terminaldan verilən seçimləri saxlayır. İlk sətirdəki `? :` ifadəsi seçim verilməyibsə eager, verilibsə birinci sözü götürür. Aşağıdakı kod Program.cs-ni tam əvəz edir:

C# - Program.cs, tam fayl

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Diagnostics;

var mode = args.Length == 0 ? "eager" : args[0];
using var connection = new SqliteConnection("Data Source=:memory:");
await connection.OpenAsync();
var sql = new List<string>();

var builder = new DbContextOptionsBuilder<CafeContext>()
    .UseSqlite(connection)
    .LogTo(sql.Add, new[] { RelationalEventId.CommandExecuted });
if (mode == "lazy") builder.UseLazyLoadingProxies();
var options = builder.Options;

await SeedData.CreateAsync(options);
sql.Clear();

await using var db = new CafeContext(options);
Order order;
if (mode == "eager")
    order = await db.Orders.Include(o => o.Items)
        .SingleAsync(o => o.Id == 42);
else
    order = await db.Orders.SingleAsync(o => o.Id == 42);

Console.WriteLine("Once SQL: " + sql.Count);
if (mode == "explicit")
    await db.Entry(order).Collection(o => o.Items).LoadAsync();

Console.WriteLine("Items: " + order.Items.Count);
Console.WriteLine("Sonra SQL: " + sql.Count);
foreach (var command in sql) Console.WriteLine(command);
```

Tam proqramdakı ayarları və sayğacı oxuyaq

SqliteConnection bu nümunədə bazaya bağlantını təmsil edir. `Data Source=:memory:` yaddaş bazası seçir. Bağlantı açıq saxlandığı müddətdə müxtəlif təzə context-lər həmin bazanı görə bilir. Proqram bitəndə bu nümunə bazası da itir.

DbContextOptionsBuilder context ayarlarını qurur. `Options` hazır ayarlar dəstini qaytarır. Eager və explicit variantlarda proxy açılmır; yalnız lazy seçimi UseLazyLoadingProxies-i işlədir.

Log icra zamanı baş verən hadisənin qeydidir. `LogTo` qeyd mesajlarını göstərilən yerə ötürür. `sql.Add` onları mətn siyahısına əlavə edir. `RelationalEventId.CommandExecuted` yalnız icra olunmuş baza əmri hadisələrini seçir.

`sql.Clear()` başlanğıc məlumatı əlavə edildikdən sonra sayğac siyahısını boşaldır. Bazadakı məlumatı silmir. `sql.Count` bundan sonra qeydə düşən əmrlərin sayıdır.

Eyni proqramda fərqli yükləmə addımı

```
eager:
    Include olan esas sorgu.

explicit:
    Esas sorgu, sonra LoadAsync.

lazy:
    Proxy aktiv esas sorgu, sonra Items getter-i.
```

Son foreach bütün SQL qeydlərini ekrana yazır. SQL sayını ölçərkən sadəcə yazdığımız LINQ sətirlərini saymırıq; həqiqətən icra edilən baza əmrlərinə baxırıq.

Üç ayrı işə salınmanın yoxlanmış çıxışı

Layihənin qovluğunda seçimləri ayrıca işlət:

Terminal

```
dotnet run -- eager
```

Çıxışın ilk üç sətri

```
Once SQL: 1
Items: 2
Sonra SQL: 1
```

Terminal

```
dotnet run -- explicit
```

Çıxışın ilk üç sətri

```
Once SQL: 1
Items: 2
Sonra SQL: 2
```

Terminal

```
dotnet run -- lazy
```

Çıxışın ilk üç sətri

```
Once SQL: 1
Items: 2
Sonra SQL: 2
```

Once SQL əsas Order sorğusundan sonrakı, Sonra SQL isə əlaqə istifadə ediləndən sonrakı ümumi saydır. Explicit və lazy-də yekun eynidir, amma əlavə SQL birində LoadAsync, digərində Items getter-i ilə başlayır. Bu üç sətirdən sonra tam SQL qeydləri də görünür.

Nəticəni kodu işlətməzdən əvvəl proqnozlaşdır

1. Təzə context, lazy bağlı, Include yoxdur. Order 42-nin Items.Count-u? Nümunədə 0 olur; əlaqə gətirilməyib. Bazada iki məhsulun olması ayrı faktdır.

2. Include ilə Order 42 alınıb. Sonra Items iki dəfə oxunur. Bu sadə single-query nümunəsində SQL sayı 1 qalır, məhsul sayı 2 olur.

3. Explicit ilə tam Items yüklənib, sonra eyni LoadAsync təkrarlanır. Bu təcrübədə əlavə SQL getmir. Qismən yükləmə işarələrini ayrıca nəzərə almaq lazımdır.

4. Üç sifarişdə lazy Items oxunur. 1 əsas + 3 əlaqə sorğusu = 4 SQL. Boş 44 əlaqəsi də yoxlanılır.

5. Select içində ItemCount seçilir. Verilmiş ifadə SQL-ə çevrilir, bir SQL ilə saylar alınır. Əvvəl tam kolleksiyaları yükləmək lazım olmur.

6. İki kolleksiya Include + AsSplitQuery. Göstərilən modeldə 3 SQL: Order, Items, Payments. Loading qərarı yenə eager-dir.

7. No-tracking entity-də lazy yükləmə mümkündürmü? Bu dərsin yoxlanmış EF Core 10.0.9 proxy variantında mümkündür. Tracking-i loading-i söndürmə vasitəsi kimi istifadə etmirik.

8. API cavabı üçün yalnız nömrə və say lazımdır. API proqramların bir-birinə müraciət etməsi üçün təqdim olunan giriş imkanlarıdır. Belə cavabda ayrıca DTO və projection məlumatı və xassə oxunuşlarının davranışını daha açıq göstərir.

Rəsmi mənbələr və sınağın sərhədləri

Bu dərsin sifariş modeli, kodları və addım izahları xüsusi hazırlanıb. Texniki davranışlar üçün əsas rəsmi mənbələr:

- Microsoft Learn - eager loading
- Microsoft Learn - explicit loading
- Microsoft Learn - lazy loading
- Microsoft Learn - single və split sorğular
- Microsoft Learn - tracking və no-tracking
- Microsoft Learn - navigation fixup
- Microsoft Learn - SQL qeydlərini almaq
- Microsoft Learn - DbContext ömrü

Əsas nümunələr **EF Core 10.0.9**, **SQLite provider 10.0.9** və göstərilən layihə ayarları ilə yoxlanıb. Tam proqramın üç variantı və əlavə loading ssenariləri real SQL əmrləri üzərində işlədilir. EF-in InMemory provider-i ilə alınmış SQL təxmini təqdim edilmir.

Mətn daxilində **fraqment** artıq hazırlanmış layihənin müəyyən yerinə əlavə olunan kod parçasıdır. Tam fayl yazılan hissələr isə göstərilən faylı bütövlükdə əvəz edir. Product və Payment əlavələri əsas nümunənin ayrıca genişləndirilməsidir.

Provider, EF versiyası, model qaydaları və əvvəl izlənmiş entity-lər SQL-i və obyektlərin hazır vəziyyətini dəyişə bilər. Buna görə dərsdə hər sayın yanında təcrübənin şərtləri verilir. Öz tətbiqində qərarı eyni qayda ilə: ehtiyac, faktiki SQL və nəticə həcmi üzərində yoxla.